

From the preceding output, we see that we can conditionally generate images by specifying their labels.

Now that we have learned about generating images using a diffusion model from scratch, we will learn about leveraging Stable Diffusion to generate images given a text prompt.

Understanding Stable Diffusion

So far, we've learned how diffusion models work. Stable Diffusion improves upon the UNet2D model by first leveraging VAE to encode an image to a lower dimension and then performing training on the down-scaled/latent space. Once the model training is done, we use a VAE decoder to get a high-resolution image. This way, training is faster as the model learns features from the latent space than from the pixel values.

The architecture of Stable Diffusion is as follows:

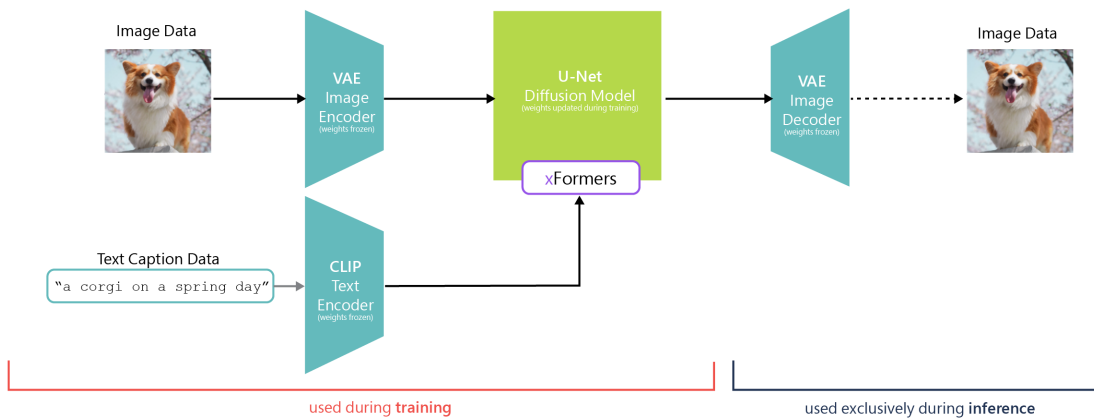


Figure 16.17: Stable Diffusion overview

The VAE encoder is a standard auto-encoder that takes an input image of shape 768x768 and returns a 96x96 image. The VAE decoder takes a 96x96 image and upscales it to 768x768.

The pre-trained Stable Diffusion U-Net model architecture is:

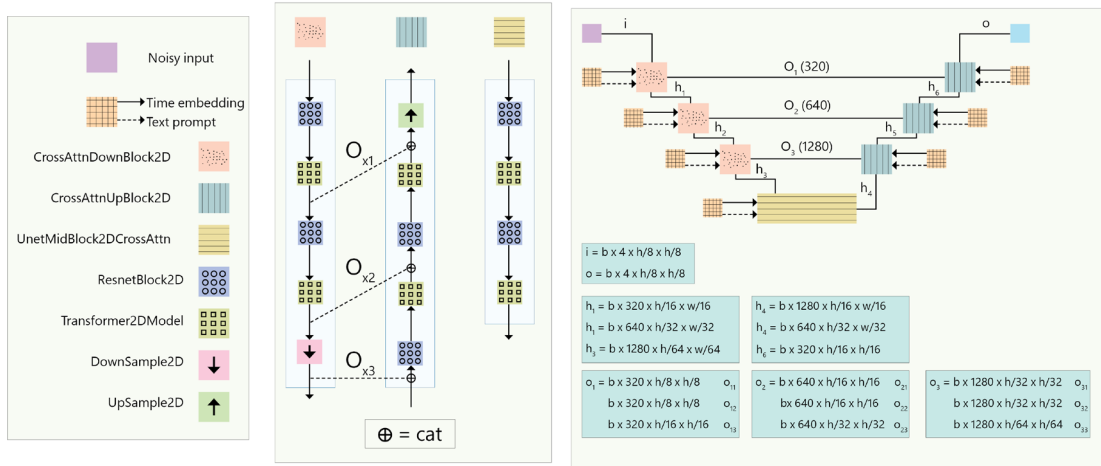


Figure 16.18: Pre-trained Stable Diffusion U-Net model architecture

In the preceding diagram, noisy input represents the output obtained from the VAE encoder. Text prompt represents the CLIP embeddings of text.

Building blocks of the Stable Diffusion model

The UNet2D architecture is central to the functionality of Stable Diffusion, an in-depth understanding of it essential for mastering the Stable Diffusion landscape. Given its complexity, characterized by numerous connections and data flows between layers, it is crucial to comprehensively understand each key layer. To achieve this, we will examine each layer through three critical components:

1. **Inputs to the Layer:** What data or tensors are fed into the layer?
2. **Transformation of Inputs:** How does the layer process or transform these inputs?
3. **Final Outputs:** What is the resulting output after the transformation? By delving into these aspects, we aim to provide a clear picture of how tensors flow through the model, thereby enabling you to master Stable Diffusion thoroughly.

Let us understand what is under the hood of the UNet2D model by downloading a pretrained Stable Diffusion model and calling its summary, like so:

```
from diffusers import StableDiffusionPipeline
from torchinfo import summary
model_id = "stabilityai/stable-diffusion-2"
pipe = StableDiffusionPipeline.from_pretrained(model_id)
summary(pipe.unet, depth=4)
```

We get the following:

```
1 =====
2 Layer (type:depth-idx).....Param #
3 =====
4 UNet2DConditionModel.....
5   └─Conv2d: 1-1.....11,840
6   └─Timesteps: 1-2.....
7   └─TimestepEmbedding: 1-3.....
8       └─LoRACompatibleLinear: 2-1.....410,880
9       └─SiLU: 2-2.....
10      └─LoRACompatibleLinear: 2-3.....1,639,680
11      └─ModuleList: 1-4.....
12          └─CrossAttnDownBlock2D: 2-4.....10,852,160
13          └─CrossAttnDownBlock2D: 2-5.....37,473,920
14          └─CrossAttnDownBlock2D: 2-6.....141,303,040
15          └─DownBlock2D: 2-7.....62,277,120
16      └─ModuleList: 1-5.....
17          └─UpBlock2D: 2-8.....162,241,280
18          └─CrossAttnUpBlock2D: 2-9.....260,296,960
19          └─CrossAttnUpBlock2D: 2-10.....72,396,160
20          └─CrossAttnUpBlock2D: 2-11.....19,302,080
21      └─UNetMidBlock2DCrossAttn: 1-6.....
22          └─ModuleList: 2-12.....35,416,320
23          └─ModuleList: 2-13.....62,277,120
24      └─GroupNorm: 1-7.....640
25      └─SiLU: 1-8.....
26      └─Conv2d: 1-9.....11,524
27 =====
28 Total params: 865,910,724
29 Trainable params: 865,910,724
30 Non-trainable params: 0
31 =====
```

Figure 16.19: UNet2D model architecture

Let's try to uncover each piece in the pipeline for an in-depth understanding of the theory behind such a successful network. We have the following high-level blocks:

- CrossAttnDownBlock2D
- CrossAttnUpBlock2D
- UNetMidBlock2DCrossAttn
- DownBlock2D
- UpBlock2D

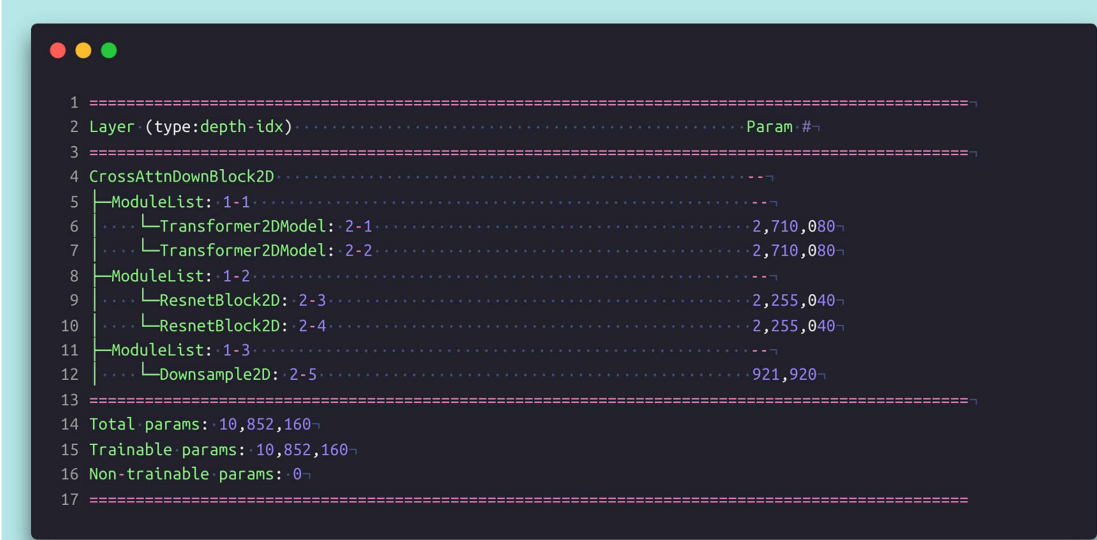
Let's look at each of these blocks in detail.

CrossAttnDownBlock2D

Running `pipe.unet.down_blocks` gives us the four down-sampling modules (i.e., the left part of UNet in *Figure 16.18*). The first three are `CrossAttnDownBlock2D` and the last one is `DownBlock2D`. On checking the summary for any of the `CrossAttnDownBlock2D`, we get:

```
Summary(pipe.unet.down_blocks[0], depth=2)
```

The preceding code results in:



```

1 =====
2 Layer (type:depth-idx).....Param #
3 =====
4 CrossAttnDownBlock2D.....
5 |ModuleList: 1-1.....
6 |... |Transformer2DModel: 2-1.....2,710,080
7 |... |Transformer2DModel: 2-2.....2,710,080
8 |ModuleList: 1-2.....
9 |... |ResnetBlock2D: 2-3.....2,255,040
10 |... |ResnetBlock2D: 2-4.....2,255,040
11 |ModuleList: 1-3.....
12 |... |Downsample2D: 2-5.....921,920
13 =====
14 Total params: 10,852,160
15 Trainable params: 10,852,160
16 Non-trainable params: 0
17 =====

```

Figure 16.20: Summary of the first down block of the unet

As you can see in *Figure 16.20*, a module is made of 3 types of blocks – `Transformer2DModel`, `ResnetBlock2D`, and `Downsample2D`.

Inspecting the forward method of the `CrossAttnDownBlock2D` class in the `github.com/huggingface/diffusers` GitHub repo at `diffusers/models/unet_2d_blocks.py`, we see that the model works with three inputs:

```
1 hidden_states: torch.FloatTensor of size 5898240 (22Mb) xE[-3.921, 3.216] μ=0.004 σ=0.351 cuda:0
2 temb: torch.FloatTensor of size 2560 (10Kb) xE[-2.014, 5.339] μ=-0.003 σ=0.256 cuda:0
3 encoder_hidden_states: torch.FloatTensor of size 157696 (0.6Mb) xE[-6.571, 13.023] μ=-0.169 σ=1.032 cuda:0
```

Figure 16.21: Inputs for `CrossAttnDownBlock2D`

As shown in the preceding screenshot, these inputs are:

- `hidden_states`, which is the embeddings corresponding to the noisy x
- `temb`, which corresponds to the embeddings of timestep
- `encoder_hidden_states`, which corresponds to the embeddings of the input text prompt

At a high level, the (simplified) order of computations within a `CrossAttnDownBlock2D` block (we have modified the code for better explainability) is provided in Figure 16.22.

```
1 def forward(
2     self,
3     hidden_states: torch.FloatTensor,
4     temb: Optional[torch.FloatTensor] = None,
5     encoder_hidden_states: Optional[torch.FloatTensor] = None,
6 ):
7     output_states = ()
8     for i, (resnet, attn) in enumerate(blocks):
9         hidden_states = resnet(hidden_states, temb)
10        hidden_states = attn(
11            hidden_states,
12            encoder_hidden_states=encoder_hidden_states,
13        )[0]
14        output_states = output_states + (hidden_states,)
15        if self.downsamplers is not None:
16            for downsampler in self.downsamplers:
17                hidden_states = downsampler(hidden_states, scale=lora_scale)
18            output_states = output_states + (hidden_states,)
19        return hidden_states, output_states
```

Figure 16.22: Forward method for `CrossAttnDownBlock2D`



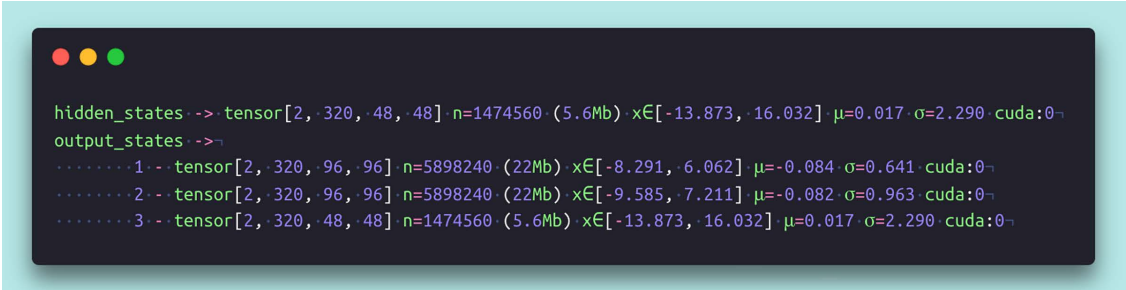
In the architecture diagram in Figure 16.20, note that `CrossAttnDownBlock2D` has ResNet and Transformer2D blocks repeated twice before performing `DownBlock2D`, and that is reflected as a for loop in Figure 16.22.

The key point to understand in the flow is that the resnet block uses temb, whereas the attn module uses encoder_hidden_states (which is related to the prompt).

Ultimately, CrossAttnDownBlock2D takes the input hidden states and passes them through its internal blocks in the following order, along with necessary temb or encoder_hidden_states tensors as inputs to resnet and attn respectively:

resnet->attn->resnet->attn->downsampler

Finally, the hidden states and intermediate hidden states are returned:



```
hidden_states-> tensor[2, 320, 48, 48] n=1474560 (5.6Mb) xE[-13.873, 16.032] μ=0.017 σ=2.290 cuda:0
output_states->
.....1-- tensor[2, 320, 96, 96] n=5898240 (22Mb) xE[-8.291, 6.062] μ=-0.084 σ=0.641 cuda:0
.....2-- tensor[2, 320, 96, 96] n=5898240 (22Mb) xE[-9.585, 7.211] μ=-0.082 σ=0.963 cuda:0
.....3-- tensor[2, 320, 48, 48] n=1474560 (5.6Mb) xE[-13.873, 16.032] μ=0.017 σ=2.290 cuda:0
```

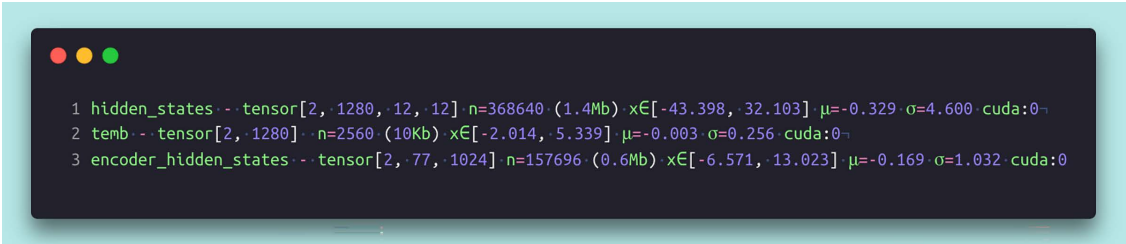
Figure 16.23: Outputs for CrossAttnDownBlock2D

Note that we obtain 320 channels as DownBlock2D gives 320 channels as output. The hidden_states are used as input for the next block, while output states are used for skip connections. This step is repeated 2 more times with an increasing number of channels before being fed to the middle block.

UNetMidBlock2DcrossAttn

This block is the bottleneck of the UNet, acting as the point where the important information is distilled as much as possible.

It takes the following inputs:



```
1 hidden_states-- tensor[2, 1280, 12, 12] n=368640 (1.4Mb) xE[-43.398, 32.103] μ=-0.329 σ=4.600 cuda:0
2 temb-- tensor[2, 1280] n=2560 (10Kb) xE[-2.014, 5.339] μ=-0.003 σ=0.256 cuda:0
3 encoder_hidden_states-- tensor[2, 77, 1024] n=157696 (0.6Mb) xE[-6.571, 13.023] μ=-0.169 σ=1.032 cuda:0
```

Figure 16.24: Inputs for UNetMidBlock2DcrossAttn

It also uses the following (simplified) forward method:

```

1 ....def forward(
2 .....self,
3 .....hidden_states: torch.FloatTensor,
4 .....temb: Optional[torch.FloatTensor] = None,
5 .....encoder_hidden_states: Optional[torch.FloatTensor] = None,
6 ....):
7 .....hidden_states = self.resnets[0](hidden_states, temb)
8 .....for attn, resnet in zip(self.attentions, self.resnets[1:]):
9 .....    hidden_states = attn(
10 .....        hidden_states,
11 .....        encoder_hidden_states=encoder_hidden_states,
12 .....    )[0]
13 .....    hidden_states = resnet(hidden_states, temb)
14 .....return hidden_states

```

Figure 16.25: forward method (simplified) for UNetMidBlock2DCrossAttn

Note that this module has an additional ResNet. The output is as follows:

```

1 tensor[2, 1280, 12, 12] n=368640 (1.4Mb) x∈[-43.307, 34.261] μ=-0.404 σ=4.882 cuda:0

```

Figure 16.26: Output for UNetMidBlock2DCrossAttn

This output is fed as `hidden_states` to the next `CrossAttnUpBlock2D` module.

CrossAttnUpBlock2D

The summary for this module is similar to that of `CrossAttnDownBlock2D`. One might think the only difference here is in the names – just Up wherever Down is present. But the important thing to consider is the fact that these blocks additionally accept the output states (`res_hidden_states_tuple` in the following screenshot) coming from the corresponding level of `CrossAttnDown2D`:

```

1. hidden_states = tensor[2, 640, 96, 96] n=11796480 (45Mb) x∈[-486.840, 68.294] μ=-0.102 σ=3.369 cuda:0
2. temb = tensor[2, 1280] n=2560 (10Kb) x∈[-2.014, 5.339] μ=-0.003 σ=0.256 cuda:0
3. res_hidden_states_tuple =
.....1.....tensor[2, 320, 96, 96] n=5898240 (22Mb) x∈[-3.706, 3.410] μ=0.003 σ=0.351 cuda:0
.....2.....tensor[2, 320, 96, 96] n=5898240 (22Mb) x∈[-8.291, 6.062] μ=-0.084 σ=0.641 cuda:0
.....3.....tensor[2, 320, 96, 96] n=5898240 (22Mb) x∈[-9.585, 7.211] μ=-0.082 σ=0.963 cuda:0
4. encoder_hidden_states = tensor[2, 77, 1024] n=157696 (0.6Mb) x∈[-6.571, 13.023] μ=-0.169 σ=1.032 cuda:0

```

Figure 16.27: CrossAttnUpBlock2D

The simplified forward method looks like the following:

```

def forward(
    self,
    hidden_states: torch.FloatTensor,
    res_hidden_states_tuple: Tuple[torch.FloatTensor, ...],
    temb: Optional[torch.FloatTensor] = None,
    encoder_hidden_states: Optional[torch.FloatTensor] = None,
):
    for resnet, attn in zip(self.resnets, self.attentions):
        # pop res hidden states
        res_hidden_states = res_hidden_states_tuple[-1]
        res_hidden_states_tuple = res_hidden_states_tuple[:-1]
        hidden_states = torch.cat([hidden_states, res_hidden_states], dim=1)
        hidden_states = resnet(hidden_states, temb, scale=lora_scale)
        hidden_states = attn(
            hidden_states,
            encoder_hidden_states=encoder_hidden_states,
            return_dict=False,
        )[0]

    if self.upsamplers is not None:
        for upsampler in self.upsamplers:
            hidden_states = upsampler(hidden_states, upsample_size, scale=lora_scale)

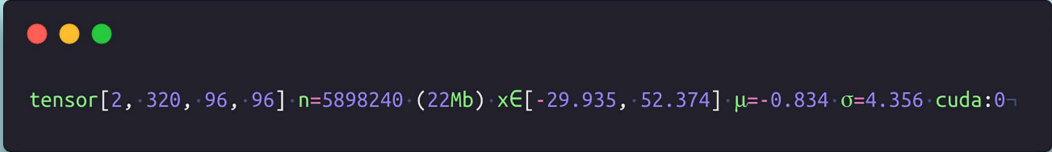
    return hidden_states

```

Figure 16.28: forward method (simplified) for CrossAttnUpBlock2D

The additional input called `res_hidden_states_tuple` is a collection of three tensors (as there are three `output_states` per down block). Each tensor is concatenated with `hidden_states` and fed to the resnet.

The output is a single tensor of hidden states:



```
tensor[2, 320, 96, 96] · n=5898240 · (22Mb) · xE[-29.935, 52.374] · μ=-0.834 · σ=4.356 · cuda:0
```

Figure 16.29: Output for `CrossAttnUpBlock2D`

Note that we get output of shape 96x96, which is the input image shape.

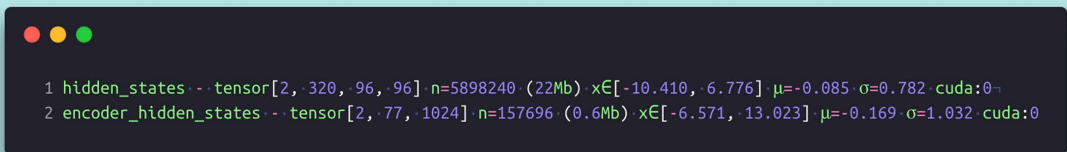
DownBlock2D, UpBlock2D

These two blocks contain only Resnet2D blocks and do not have attention modules. The activity to find the inputs, outputs, and forward of these blocks is left as an exercise for the reader. You are encouraged to go through github.com/huggingface/diffusers and search for the class definitions of `DownBlock2D` and `UpBlock2D`.

Transformer2DModel

This is a standard self-attention-based transformer encoder module that has an additional responsibility to convert 2D inputs to 1D during the input phase and back to 2D outputs at the end.

Here are the inputs:



```
1 hidden_states · · tensor[2, 320, 96, 96] · n=5898240 · (22Mb) · xE[-10.410, 6.776] · μ=-0.085 · σ=0.782 · cuda:0
2 encoder_hidden_states · · tensor[2, 77, 1024] · n=157696 · (0.6Mb) · xE[-6.571, 13.023] · μ=-0.169 · σ=1.032 · cuda:0
```

Figure 16.30: Input of `Transformer2DModel`

A (simplified) forward method for the module is given below:

```

1  def forward(
2  ... self,
3  ... hidden_states: torch.Tensor,
4  ... encoder_hidden_states: Optional[torch.Tensor] = None,
5  ... ):
6  ... batch, _, height, width = hidden_states.shape
7  ... residual = hidden_states
8  ... hidden_states = self.norm(hidden_states)
9  ... hidden_states = self.proj_in(hidden_states) # nn.Conv2D
10 ... inner_dim = hidden_states.shape[1]
11 ... hidden_states = hidden_states.permute(0, 2, 3, 1).reshape(batch, height * width, inner_dim)
12 ... for block in self.transformer_blocks:
13 ...     hidden_states = block(
14 ...         hidden_states,
15 ...         attention_mask=attention_mask,
16 ...         encoder_hidden_states=encoder_hidden_states,
17 ...     )
18 ... hidden_states = self.proj_out(hidden_states) # nn.Conv2D
19 ... hidden_states = hidden_states.reshape(batch, height, width, inner_dim).permute(0, 3, 1, 2).contiguous()
20 ... output = hidden_states + residual
21 ... return Transformer2DModelOutput(sample=output)

```

Figure 16.31: forward method (simplified) for Transformer2DModel

The input is stored as residual at first and then the hidden_states tensor is projected through a convolution layer (self.proj_in in the preceding code, resulting in a shape of 2x786x96x96). The height and width dimensions are rearranged in the shape [bs, h x w, channels]. Once this is passed through the transformer blocks, the input and output shapes are the same. The output of this step is reshaped and permuted to (bs x h x w x channels) and is passed through a convolution to get the original dimensions:

```

1  tensor[2, 320, 96, 96] · n=5898240 · (22Mb) · x ∈ [-30.745, 52.179] · μ = -0.837 · σ = 4.354 · cuda:0

```

Figure 16.32: Output of Transformer2DModel

ResnetBlock2D

This block is nothing different from a standard resnet, except for the acceptance of a new temb variable. Here are the inputs:

```
1 hidden_states = tensor[2, 320, 96, 96] · n=5898240 (22Mb) · x∈[-3.619, 3.284] · μ=0.004 · σ=0.348 · cuda:0
2 temb = tensor[2, 1280] · n=2560 (10Kb) · x∈[-2.014, -5.339] · μ=-0.003 · σ=0.256 · cuda:0
```

Figure 16.33: Input for ResnetBlock2D

Here's the forward method:

```
1 def forward(
2     self,
3     input_tensor: torch.FloatTensor,
4     temb: torch.FloatTensor,
5 ):
6     hidden_states = input_tensor
7     hidden_states = self.norm1(hidden_states)
8     hidden_states = self.nonlinearity(hidden_states)
9     hidden_states = self.conv1(hidden_states)
10    if self.time_emb_proj is not None:
11        if not self.skip_time_act:
12            temb = self.nonlinearity(temb)
13            temb = self.time_emb_proj(temb)[:,:,: None, None]
14            hidden_states = hidden_states + temb
15            hidden_states = self.nonlinearity(hidden_states)
16            hidden_states = self.dropout(hidden_states)
17            hidden_states = self.conv2(hidden_states, scale) if not USE_PEFT_BACKEND else self.conv2(hidden_states)
18            output_tensor = (input_tensor + hidden_states)
19            return output_tensor
```

Figure 16.34: forward method for ResnetBlock2D

Here, `self.time_emb_proj` is a linear layer making sure the channel's dimension for `temb` is the same as `hidden_states`.

The output is simply the same shape as `hidden_states`:

```
1 tensor[2, 320, 96, 96] · n=5898240 (22Mb) · x∈[-7.879, 5.240] · μ=-0.017 · σ=0.642 · cuda:0
```

Figure 16.35: Output for ResnetBlock2D

We now take the 320x96x96 dimensional output and pass it through the VAE decoder to fetch the output image.

Now that we understand the working details of the Stable Diffusion model, let us go ahead and leverage it to take a text prompt and convert it to an image.

Implementing Stable Diffusion

To generate images given a text prompt, we will leverage the `diffusers` library built by the Hugging Face team. We will use the following code to do this:



This code is available in the `Conditional_Diffuser_training.ipynb` file in the `Chapter16` folder in the GitHub repo at <https://bit.ly/mcnp-2e>. Do be sure to run the code from the notebook and refer to the following explanations to understand the different steps.

1. Log in to Hugging Face and provide the auth token:

```
!huggingface-cli login
```

2. Install the required packages:

```
!pip install -q accelerate diffusers
```

3. Import the required libraries:

```
from torch import autocast
from diffusers import DiffusionPipeline
```

4. Define the generator and set the seed to ensure reproducibility:

```
# Setting a seed would ensure reproducibility of the experiment.
generator = torch.Generator(device="cuda").manual_seed(42)
```

5. As we have seen in previous chapters, huggingface (and by extension diffusers) wraps models in the form of pipelines that are easy to use for the end user. Let's define the Hugging Face pipeline:

```
# Define the Stable Diffusion pipeline
pipeline = DiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4",
    torch_dtype=torch.float16,
)

# Set the device for the pipeline
pipeline = pipeline.to("cuda")
```

6. Pass a text prompt through the pipeline defined above:

```
prompt = "baby in superman dress"  
image = pipeline(prompt, generator=generator)
```

7. Visualize the image:

```
image.images[0]
```

This gives the following output:



Figure 16.36: Image generated by Stable Diffusion

Note that the above generated image has multiple issues – multiple babies are generated, legs do not seem aligned, etc. Can we do better?

8. Stable Diffusion has an XL variant that has been trained on more data while generating higher-resolution images (1024x1024), due to which the chances of such mistakes are lower. Let's replace the base model with the XL version of it, as follows:

```
# Define the Stable Diffusion XL pipeline  
pipeline = DiffusionPipeline.from_pretrained(  
    "stabilityai/stable-diffusion-xl-base-1.0",  
    torch_dtype=torch.float16,  
)  
  
# Set the device for the pipeline  
pipeline = pipeline.to("cuda")
```

9. Pass the prompt through the pipeline that we just defined:

```
prompt = "baby in superman dress"  
image = pipe(prompt, generator=generator)
```

The output is:



Figure 16.37: Output of SDXL

Note that, the image generated using the XL pipeline is much better than the image generated using the base pipeline. However, can we get even better?

10. Prompt engineering comes to the rescue in this scenario. We can modify our prompt as follows:

```
prompt = "baby in superman dress, photorealistic, cinematic"
```

The preceding prompt, when passed through the XL pipeline, results in an output as follows:



Figure 16.38: Realistic output of SDXL

Note that, by adding words like “photorealistic” and “cinematic”, we were able to generate an image that looks more dramatic than the plain one generated earlier.

Summary

In this chapter, we learned how CLIP helps in aligning embeddings of both text and images. We then gained an understanding of how to leverage the SAM to perform segmentation on any image. Next, we learned about speeding up the SAM using FastSAM. Finally, we learned about leveraging diffusion models to generate images both unconditionally and conditionally given a prompt.

We covered sending different modalities of prompts to the segment-anything model, tracking objects using the SAM, and combining multiple modalities using ImageBind in the associated GitHub repository.

With this knowledge, you can leverage the foundational models on your data/tasks with very limited/no training data points, such as training/leveraging models for the segmentation/object detection tasks that we learned about in *Chapters 7 to 9* with minimal/no data.

In the next chapter, you will learn about tweaking diffusion models further to generate images of interest to you.

Questions

1. How are text and images represented in the same domain using CLIP?
2. How are different types of tokens, such as point tokens, bounding box tokens, and text tokens, calculated in Segment Anything architecture?
3. How do diffusion models work?
4. What makes Stable Diffusion different from normal diffusion?
5. What is the difference between Stable Diffusion and the SDXL model?

Learn more on Discord

Join our community’s Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

